JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class SocketChannel

java.lang.Object
  java.nio.channels.spi.AbstractInterruptibleChannel
    java.nio.channels.SelectableChannel
      java.nio.channels.spi.AbstractSelectableChannel
        java.nio.channels.SocketChannel

All Implemented Interfaces:

`Closeable`, `AutoCloseable`, `ByteChannel`, `Channel`, `GatheringByteChannel`, `InterruptibleChannel`, `NetworkChannel`, `ReadableByteChannel`, `ScatteringByteChannel`, `WritableByteChannel`

```
public abstract class SocketChannel
extends AbstractSelectableChannel
implements ByteChannel, ScatteringByteChannel, GatheringByteChannel,
NetworkChannel
```

A selectable channel for stream-oriented connecting sockets.

A socket channel is created by invoking one of the open methods of this class. The no-arg `open` method opens a socket channel for an *Internet protocol* socket. The `open(ProtocolFamily)` method is used to open a socket channel for a socket of a specified protocol family. It is not possible to create a channel for an arbitrary, pre-existing socket. A newly-created socket channel is open but not yet connected. An attempt to invoke an I/O operation upon an unconnected channel will cause a `NotYetConnectedException` to be thrown. A socket channel can be connected by invoking its `connect` method; once connected, a socket channel remains connected until it is closed. Whether or not a socket channel is connected may be determined by invoking its `isConnected` method.

Socket channels support *non-blocking connection:* A socket channel may be created and the process of establishing the link to the remote socket may be initiated via the `connect` method for later completion by the `finishConnect` method. Whether or not a connection operation is in progress may be determined by invoking the `isConnectionPending` method.

Socket channels support *asynchronous shutdown,* which is similar to the asynchronous close operation specified in the `Channel` class. If the input side of a socket is shut down by one thread while another thread is blocked in a read operation on the socket's channel, then the read operation in the blocked thread will complete without reading any bytes and will return `-1`. If the output side of a socket is shut down by one thread while another thread is blocked in a write operation on the socket's channel, then the blocked thread will receive an `AsynchronousCloseException`.

Socket options are configured using the `setOption` method. Socket channels for *Internet protocol* sockets support following options:

Option Name	Description
`SO_SNDBUF`	The size of the socket send buffer
`SO_RCVBUF`	The size of the socket receive buffer
`SO_KEEPALIVE`	Keep connection alive
`SO_REUSEADDR`	Re-use address
`SO_LINGER`	Linger on close if data is present (when configured in blocking mode only)
`TCP_NODELAY`	Disable the Nagle algorithm

Socket channels for *Unix domain* sockets support:

Option Name	Description
`SO_SNDBUF`	The size of the socket send buffer
`SO_RCVBUF`	The size of the socket receive buffer
`SO_LINGER`	Linger on close if data is present (when configured in blocking mode only)

Additional (implementation specific) options may also be supported.

Socket channels are safe for use by multiple concurrent threads. They support concurrent reading and writing, though at most one thread may be reading and at most one thread may be writing at any given time. The `connect` and `finishConnect` methods are mutually synchronized against each other, and an attempt to initiate a read or write operation while an invocation of one of these methods is in progress will block until that invocation is complete.

Since:

1.4

Constructor Summary

Constructors

Modifier	Constructor	Description
`protected`	`SocketChannel (SelectorProvider provid`	Initializes a new instance of this class.

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
abstract SocketChannel	**bind** (SocketAddress local)	Binds the channel's socket to a local address.
abstract boolean	**connect** (SocketAddress remote)	Connects this channel's socket.
abstract boolean	**finishConnect**()	Finishes the process of connecting a socket channel.
abstract SocketAddress	**getLocalAddress**()	Returns the socket address that this channel's socket is bound to.
abstract SocketAddress	**getRemoteAddress**()	Returns the remote address to which this channel's socket is connected.
abstract boolean	**isConnected**()	Tells whether or not this channel's network socket is connected.
abstract boolean	**isConnectionPending**()	Tells whether or not a connection operation is in progress on this channel.
static SocketChannel	**open**()	Opens a socket channel for an *Internet protocol* socket.
static SocketChannel	**open** (ProtocolFamily family)	Opens a socket channel.
static SocketChannel	**open** (SocketAddress remote)	Opens a socket channel and connects it to a remote address.
abstract int	**read**(ByteBuffer dst)	

		Reads a sequence of bytes from this channel into the given buffer.
final long	**read**(ByteBuffer[] dsts)	Reads a sequence of bytes from this channel into the given buffers.
abstract long	**read**(ByteBuffer[] dsts, int offset, int length)	Reads a sequence of bytes from this channel into a subsequence of the given buffers.
abstract <T> SocketChannel	**setOption**(SocketOption<T> name, T value)	Sets the value of a socket option.
abstract SocketChannel	**shutdownInput**()	Shutdown the connection for reading without closing the channel.
abstract SocketChannel	**shutdownOutput**()	Shutdown the connection for writing without closing the channel.
abstract Socket	**socket**()	Retrieves a socket associated with this channel.
final int	**validOps**()	Returns an operation set identifying this channel's supported operations.
abstract int	**write**(ByteBuffer src)	Writes a sequence of bytes to this channel from the given buffer.
final long	**write**(ByteBuffer[] srcs)	Writes a sequence of bytes to this channel from the given buffers.
abstract long	**write**(ByteBuffer[] srcs, int offset, int length)	Writes a sequence of bytes to this channel from a subsequence of the given buffers.

Methods declared in class AbstractSelectableChannel

blockingLock, configureBlocking, implCloseChannel, implCloseSelectableChannel, implConfigureBlocking, isBlocking, isRegistered, keyFor, provider, register

Methods declared in class SelectableChannel

register

Methods declared in class AbstractInterruptibleChannel

begin, close, end, isOpen

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Methods declared in interface NetworkChannel

getOption, supportedOptions

Constructor Details

SocketChannel

```
protected SocketChannel(SelectorProvider provider)
```

Initializes a new instance of this class.

Parameters:

`provider` - The provider that created this channel

Method Details

open

```
public static SocketChannel open()
                           throws IOException
```

Opens a socket channel for an *Internet protocol* socket.

The new channel is created by invoking the `openSocketChannel` method of the system-wide default `SelectorProvider` object.

Returns:

A new socket channel

Throws:

`IOException` - If an I/O error occurs

See Also:

java.net.preferIPv4Stack system property

open

```
public static SocketChannel open(ProtocolFamily family)
                          throws IOException
```

Opens a socket channel. The `family` parameter specifies the `protocol family` of the channel's socket.

The new channel is created by invoking the `openSocketChannel(ProtocolFamily)` method of the system-wide default. `SelectorProvider` object.

Parameters:

`family` - The protocol family

Returns:

A new socket channel

Throws:

`UnsupportedOperationException` - If the specified protocol family is not supported. For example, suppose the parameter is specified as `StandardProtocolFamily.INET6` but IPv6 is not enabled on the platform.

`IOException` - If an I/O error occurs

Since:

15

See Also:

java.net.preferIPv4Stack system property

open

```
public static SocketChannel open(SocketAddress remote)
                          throws IOException
```

Opens a socket channel and connects it to a remote address.

If the remote address is an `InetSocketAddress` then this method works as if by invoking the `open()` method, invoking the `connect` method upon the resulting socket channel, passing it `remote`, and then returning that channel.

If the remote address is a `UnixDomainSocketAddress` then this works by invoking the `open(ProtocolFamily)` method with `StandardProtocolFamily.UNIX` as parameter, invoking the `connect` method upon the resulting socket channel, passing it `remote`, then returning that channel.

Parameters:

`remote` - The remote address to which the new channel is to be connected

Returns:

A new, and connected, socket channel

Throws:

`AsynchronousCloseException` - If another thread closes this channel while the connect operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the connect operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`UnresolvedAddressException` - If the given remote address is an InetSocketAddress that is not fully resolved

`UnsupportedAddressTypeException` - If the type of the given remote address is not supported

`IOException` - If some other I/O error occurs

See Also:

java.net.preferIPv4Stack system property

validOps

```
public final int validOps()
```

Returns an operation set identifying this channel's supported operations.

Socket channels support connecting, reading, and writing, so this method returns (`SelectionKey.OP_CONNECT` | `SelectionKey.OP_READ` | `SelectionKey.OP_WRITE`).

Specified by:

`validOps` in class `SelectableChannel`

Returns:

The valid-operation set

bind

```
public abstract SocketChannel bind(SocketAddress local)
                            throws IOException
```

Binds the channel's socket to a local address.

This method is used to establish an association between the socket and a local address. For *Internet Protocol* sockets, once an association is established then the socket remains bound until the channel is closed. If the `local` parameter has the value `null` then the socket will be bound to an address that is assigned automatically.

Specified by:

`bind` in interface `NetworkChannel`

API Note:

Binding a socket channel to a *Unix Domain* socket creates a file corresponding to the file path in the `UnixDomainSocketAddress`. This file persists after the channel is closed, and must be removed before another socket can bind to the same name. If a socket channel to a Unix Domain socket is *implicitly* bound by connecting it without calling bind first, then its socket is unnamed with no corresponding socket file in the file-system. If a socket channel to a Unix Domain socket is *automatically* bound by calling `bind(null)` this results in an unnamed socket also.

Implementation Note:

Each platform enforces an implementation specific maximum length for the name of a *Unix Domain* socket. This limitation is enforced when a channel is bound. The maximum length is typically close to and generally not less than 100 bytes.

Parameters:

`local` - The address to bind the socket, or `null` to bind the socket to an automatically assigned socket address

Returns:

This channel

Throws:

`ConnectionPendingException` - If a non-blocking connect operation is already in progress on this channel

`AlreadyBoundException` - If the socket is already bound

`UnsupportedAddressTypeException` - If the type of the given address is not supported

`ClosedChannelException` - If the channel is closed

IOException - If some other I/O error occurs

Since:

1.7

See Also:

NetworkChannel.getLocalAddress()

setOption

```
public abstract <T> SocketChannel setOption(SocketOption<T> name,
                                            T value)
                                     throws IOException
```

Description copied from interface: NetworkChannel

Sets the value of a socket option.

Specified by:

setOption in interface NetworkChannel

Type Parameters:

T - The type of the socket option value

Parameters:

name - The socket option

value - The value of the socket option. A value of null may be a valid value for some socket options.

Returns:

This channel

Throws:

UnsupportedOperationException - If the socket option is not supported by this channel

IllegalArgumentException - If the value is not a valid value for this socket option

ClosedChannelException - If this channel is closed

IOException - If an I/O error occurs

Since:

1.7

See Also:

StandardSocketOptions

shutdownInput

```
public abstract SocketChannel shutdownInput()
                                       throws IOException
```

Shutdown the connection for reading without closing the channel.

Once shutdown for reading then further reads on the channel will return `-1`, the end-of-stream indication. If the input side of the connection is already shutdown then invoking this method has no effect.

Returns:

The channel

Throws:

`NotYetConnectedException` - If this channel is not yet connected

`ClosedChannelException` - If this channel is closed

`IOException` - If some other I/O error occurs

Since:

1.7

shutdownOutput

```
public abstract SocketChannel shutdownOutput()
                                        throws IOException
```

Shutdown the connection for writing without closing the channel.

Once shutdown for writing then further attempts to write to the channel will throw `ClosedChannelException`. If the output side of the connection is already shutdown then invoking this method has no effect.

Returns:

The channel

Throws:

`NotYetConnectedException` - If this channel is not yet connected

`ClosedChannelException` - If this channel is closed

`IOException` - If some other I/O error occurs

Since:

1.7

socket

```
public abstract Socket socket()
```

Retrieves a socket associated with this channel.

Returns:

A socket associated with this channel

Throws:

`UnsupportedOperationException` - If the channel's socket is not an *Internet protocol* socket

isConnected

```
public abstract boolean isConnected()
```

Tells whether or not this channel's network socket is connected.

Returns:

`true` if, and only if, this channel's network socket is `open` and connected

isConnectionPending

```
public abstract boolean isConnectionPending()
```

Tells whether or not a connection operation is in progress on this channel.

Returns:

`true` if, and only if, a connection operation has been initiated on this channel but not yet completed by invoking the `finishConnect` method

connect

```
public abstract boolean connect(SocketAddress remote)
                         throws IOException
```

Connects this channel's socket.

If this channel is in non-blocking mode then an invocation of this method initiates a non-blocking connection operation. If the connection is established immediately, as can happen with a local connection, then this method returns `true`. Otherwise this method returns `false` and the connection operation must later be completed by invoking the `finishConnect` method.

If this channel is in blocking mode then an invocation of this method will block until the connection is established or an I/O error occurs.

This method may be invoked at any time. If a read or write operation upon this channel is invoked while an invocation of this method is in progress then that operation will first block until this invocation is complete. If a connection attempt is initiated but fails, that is, if an invocation of this method throws a checked exception, then the channel will be closed.

Parameters:

`remote` - The remote address to which this channel is to be connected

Returns:

`true` if a connection was established, `false` if this channel is in non-blocking mode and the connection operation is in progress

Throws:

`AlreadyConnectedException` - If this channel is already connected

`ConnectionPendingException` - If a non-blocking connection operation is already in progress on this channel

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the connect operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the connect operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`UnresolvedAddressException` - If the given remote address is an InetSocketAddress that is not fully resolved

`UnsupportedAddressTypeException` - If the type of the given remote address is not supported

`IOException` - If some other I/O error occurs

finishConnect

```
public abstract boolean finishConnect()
                                throws IOException
```

Finishes the process of connecting a socket channel.

A non-blocking connection operation is initiated by placing a socket channel in non-blocking mode and then invoking its `connect` method. Once the connection is established, or the attempt has failed, the socket channel will become connectable and this method may be invoked to complete the connection sequence. If the connection operation failed then invoking this method will cause an appropriate `IOException` to be thrown.

If this channel is already connected then this method will not block and will immediately return `true`. If this channel is in non-blocking mode then this method will return `false` if the connection process is not yet complete. If this channel is in blocking mode then this method will block until the connection either completes or fails, and will always either return `true` or throw a checked exception describing the failure.

This method may be invoked at any time. If a read or write operation upon this channel is invoked while an invocation of this method is in progress then that operation will first block until this invocation is complete. If a connection attempt fails, that is, if an invocation of this method throws a checked exception, then the channel will be closed.

Returns:

`true` if, and only if, this channel's socket is now connected

Throws:

`NoConnectionPendingException` - If this channel is not connected and a connection operation has not been initiated

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the connect operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the connect operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

getRemoteAddress

```
public abstract SocketAddress getRemoteAddress()
                                          throws IOException
```

Returns the remote address to which this channel's socket is connected.

Where the channel's socket is bound and connected to an *Internet Protocol* socket address then the return value is of type `InetSocketAddress`.

Where the channel's socket is bound and connected to a *Unix Domain* socket address, the returned address is a `UnixDomainSocketAddress`.

Returns:

The remote address; `null` if the channel's socket is not connected

Throws:

`ClosedChannelException` - If the channel is closed

`IOException` - If an I/O error occurs

Since:

1.7

read

```
public abstract int read(ByteBuffer dst)
                  throws IOException
```

Description copied from interface: `ReadableByteChannel`

Reads a sequence of bytes from this channel into the given buffer.

An attempt is made to read up to *r* bytes from the channel, where *r* is the number of bytes remaining in the buffer, that is, `dst.remaining()`, at the moment this method is invoked.

Suppose that a byte sequence of length *n* is read, where `0` <= *n* <= *r*. This byte sequence will be transferred into the buffer so that the first byte in the sequence is at index *p* and the last byte is at index *p* + *n* - `1`, where *p* is the buffer's position at the moment this method is invoked. Upon return the buffer's position will be equal to *p* + *n*; its limit will not have changed.

A read operation might not fill the buffer, and in fact it might not read any bytes at all. Whether or not it does so depends upon the nature and state of the channel. A socket channel in non-blocking mode, for example, cannot read any more bytes than are immediately available from the socket's input buffer; similarly, a file channel cannot read any more bytes than remain in the file. It is guaranteed, however, that if a channel is in blocking mode and there is at least one byte remaining in the buffer then this method will block until at least one byte is read.

This method may be invoked at any time. If another thread has already initiated a read operation upon this channel, however, then an invocation of this method will block until the first operation is complete.

Specified by:

`read` in interface `ReadableByteChannel`

Parameters:

`dst` - The buffer into which bytes are to be transferred

Returns:

The number of bytes read, possibly zero, or `-1` if the channel has reached end-of-stream

Throws:

`NotYetConnectedException` - If this channel is not yet connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the read operation is in progress

ClosedByInterruptException - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupted status

IOException - If some other I/O error occurs

read

```
public abstract long read(ByteBuffer[] dsts,
                          int offset,
                          int length)
                   throws IOException
```

Description copied from interface: ScatteringByteChannel

Reads a sequence of bytes from this channel into a subsequence of the given buffers.

An invocation of this method attempts to read up to *r* bytes from this channel, where *r* is the total number of bytes remaining in the specified subsequence of the given buffer array, that is,

```
dsts[offset].remaining()
    + dsts[offset+1].remaining()
    + ... + dsts[offset+length-1].remaining()
```

at the moment that this method is invoked.

Suppose that a byte sequence of length *n* is read, where `0` <= *n* <= *r*. Up to the first `dsts[offset].remaining()` bytes of this sequence are transferred into buffer `dsts[offset]`, up to the next `dsts[offset+1].remaining()` bytes are transferred into buffer `dsts[offset+1]`, and so forth, until the entire byte sequence is transferred into the given buffers. As many bytes as possible are transferred into each buffer, hence the final position of each updated buffer, except the last updated buffer, is guaranteed to be equal to that buffer's limit.

This method may be invoked at any time. If another thread has already initiated a read operation upon this channel, however, then an invocation of this method will block until the first operation is complete.

Specified by:

read in interface ScatteringByteChannel

Parameters:

`dsts` - The buffers into which bytes are to be transferred

`offset` - The offset within the buffer array of the first buffer into which bytes are to be transferred; must be non-negative and no larger than `dsts.length`

`length` - The maximum number of buffers to be accessed; must be non-negative and no larger than `dsts.length - offset`

Returns:

The number of bytes read, possibly zero, or `-1` if the channel has reached end-of-stream

Throws:

`NotYetConnectedException` - If this channel is not yet connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the read operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

read

```
public final long read(ByteBuffer[] dsts)
                throws IOException
```

Description copied from interface: `ScatteringByteChannel`

Reads a sequence of bytes from this channel into the given buffers.

An invocation of this method of the form `c.read(dsts)` behaves in exactly the same manner as the invocation

```
    c.read(dsts, 0, dsts.length);
```

Specified by:

`read` in interface `ScatteringByteChannel`

Parameters:

`dsts` - The buffers into which bytes are to be transferred

Returns:

The number of bytes read, possibly zero, or `-1` if the channel has reached end-of-stream

Throws:

`NotYetConnectedException` - If this channel is not yet connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the read operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

write

```
public abstract int write(ByteBuffer src)
                  throws IOException
```

Description copied from interface: `WritableByteChannel`

Writes a sequence of bytes to this channel from the given buffer.

An attempt is made to write up to *r* bytes to the channel, where *r* is the number of bytes remaining in the buffer, that is, `src.remaining()`, at the moment this method is invoked.

Suppose that a byte sequence of length *n* is written, where `0` <= *n* <= *r*. This byte sequence will be transferred from the buffer starting at index *p*, where *p* is the buffer's position at the moment this method is invoked; the index of the last byte written will be *p* + *n* - 1. Upon return the buffer's position will be equal to *p* + *n*; its limit will not have changed.

For many types of channels, a write operation will return only after writing all of the *r* requested bytes. Some types of channels, depending upon their state, may write only some of the bytes or possibly none at all. A socket channel in non-blocking mode, for example, cannot write any more bytes than are free in the socket's output buffer. The write method may need to be invoked more than once to ensure that all remaining bytes are written.

This method may be invoked at any time. If another thread has already initiated a write operation upon this channel, however, then an invocation of this method will block until the first operation is complete.

Specified by:

`write` in interface `WritableByteChannel`

Parameters:

`src` - The buffer from which bytes are to be retrieved

Returns:

The number of bytes written, possibly zero

Throws:

`NotYetConnectedException` - If this channel is not yet connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the write operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the write operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

write

```
public abstract long write(ByteBuffer[] srcs,
                           int offset,
                           int length)
                    throws IOException
```

Description copied from interface: `GatheringByteChannel`

Writes a sequence of bytes to this channel from a subsequence of the given buffers.

An attempt is made to write up to *r* bytes to this channel, where *r* is the total number of bytes remaining in the specified subsequence of the given buffer array, that is,

```
srcs[offset].remaining()
    + srcs[offset+1].remaining()
    + ... + srcs[offset+length-1].remaining()
```

at the moment that this method is invoked.

Suppose that a byte sequence of length *n* is written, where `0` <= *n* <= *r*. Up to the first `srcs[offset].remaining()` bytes of this sequence are written from buffer `srcs[offset]`, up to the next `srcs[offset+1].remaining()` bytes are written from buffer `srcs[offset+1]`, and so forth, until the entire byte sequence is written. As many bytes as possible are written from each buffer, hence the final position of each updated buffer, except the last updated buffer, is guaranteed to be equal to that buffer's limit.

For many types of channels, a write operation will return only after writing all of the *r* requested bytes. Some types of channels, depending upon their state, may write only some of the bytes or possibly none at all. A socket channel in non-blocking mode, for example, cannot write any more bytes than are free in the socket's output buffer. The write method may need to be invoked more than once to ensure that all remaining bytes are written.

This method may be invoked at any time. If another thread has already initiated a write operation upon this channel, however, then an invocation of this method will block until the first operation is complete.

Specified by:

`write` in interface `GatheringByteChannel`

Parameters:

`srcs` - The buffers from which bytes are to be retrieved

`offset` - The offset within the buffer array of the first buffer from which bytes are to be retrieved; must be non-negative and no larger than `srcs.length`

`length` - The maximum number of buffers to be accessed; must be non-negative and no larger than `srcs.length - offset`

Returns:

The number of bytes written, possibly zero

Throws:

`NotYetConnectedException` - If this channel is not yet connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the write operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the write operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

write

```
public final long write(ByteBuffer[] srcs)
                 throws IOException
```

Description copied from interface: `GatheringByteChannel`

Writes a sequence of bytes to this channel from the given buffers.

An invocation of this method of the form `c.write(srcs)` behaves in exactly the same manner as the invocation

```
    c.write(srcs, 0, srcs.length);
```

Specified by:

`write` in interface `GatheringByteChannel`

Parameters:

`srcs` - The buffers from which bytes are to be retrieved

Returns:

The number of bytes written, possibly zero

Throws:

`NotYetConnectedException` - If this channel is not yet connected

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the write operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the write operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

getLocalAddress

```
public abstract SocketAddress getLocalAddress()
                                        throws IOException
```

Returns the socket address that this channel's socket is bound to.

Where the channel is `bound` to an Internet Protocol socket address then the return value from this method is of type `InetSocketAddress`.

Where the channel is bound to a Unix Domain socket address, the socket address is a `UnixDomainSocketAddress`.

Specified by:

`getLocalAddress` in interface `NetworkChannel`

Returns:

The `SocketAddress` that the socket is bound to; `null` if the channel's socket is not bound

Throws:

`ClosedChannelException` - If the channel is closed

`IOException` - If an I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.